Institut Supérieur d'Informatique et de Mathématiques de Monastir

Cours :Architecture des Systèmes d'Entreprise (ESA)

Chapitre2: L'Architecture Logicielle d'un Système d'Entreprise

Enseignante: Dr. Aljia BOUZIDI
aljia.bouzidi@gmail.com

3ème Licence en Sciences d'Informatique

Année Universitaire :2025-2026

Objectifs

- Appréhender un Système d'entreprise à haut niveau, dans sa globalité
- Comprendre les flux de communication entre les différents composants d'une entreprise
- Comprendre l'architecture logicielle
- Connaitre les différentes vues d'u système d'entreprise

Contenu du chapitre 2

1. **Partie 1:** Mise en Contexte
2. **Partie 2:** Vues Architecturales



Partie 1: Mise en Contexte

Contenu de la Partie 1

1. Architecture Informatique
2. C'est Quoi Architecture Logicielle?
3. Composants Logiciels
4. Connecteurs
5. Une succession d'Abstractions
6. Positionnement
7. Intérêt
8. Architecture Vs Conception
9. Concevoir l'Architecture
10. Présentation d'un Architecte Logiciel

Architecture Informatique

L'**architecture informatique** définit la structuration d'un système informatique (i.e. **matériel** et **logiciel**) en termes de composants et d'organisation de ses fonctions

C’est Quoi Architecture Logicielle? (1/4)

L’**architecture logicielle (Software architecture)**

- décrit d’une manière symbolique et schématique la structure ou les structures du système
- Elle inclut:
  - Les **composantes logicielles** (d’un ou de plusieurs programmes informatiques)
  - Les p**ropriétés** exposées de ces composantes
  - Les **interrelations** et les in**teractions** entre celles-ci



C'est Quoi Architecture Logicielle? (2/4)

L'**architecture logicielle = Abstraction logiciel**

- Collection de **composantes logicielles:**
  - Client, serveur, filtre, couche, contrôleur, agent
- **Topologie** de composantes (interrelations)**:**
  - Agrégation, classification, généralisation, association
- **Interactions** entre les composantes**:**
  - **Connecteurs**:
    - Propriétés des composantes
  - **Mode d'interaction**:
    - Partage de mémoire, synchronisation

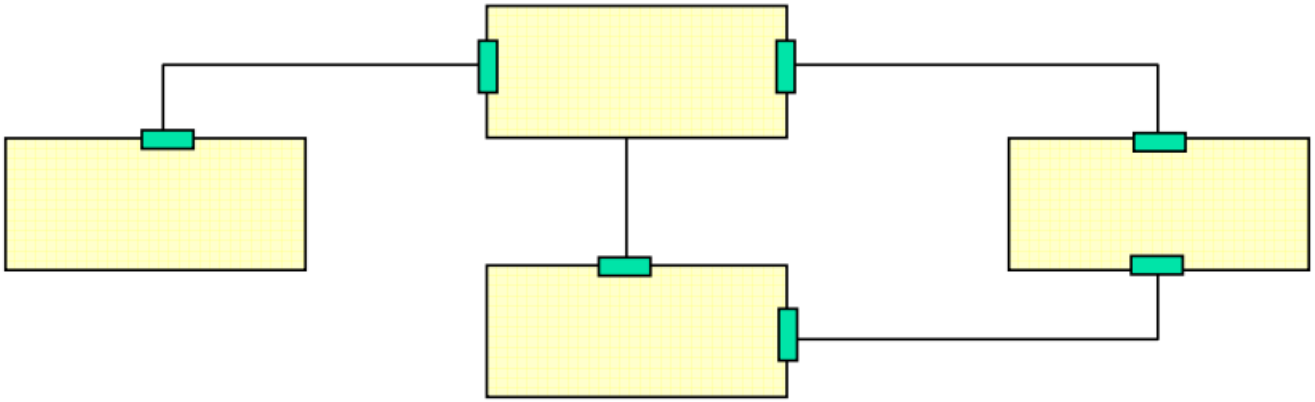
C’est Quoi Architecture Logicielle? (3/4)

- Composantes
  - Orienté Objet
    - Un module (package, librairie)
    - Un groupe de classes
    - Encapsulation des méthodes
  - Procédurale
    - Un groupe de fonctions et de structure de données
- Connecteurs:
  - Les signatures des composantes
    - Interface et services offerts par une composantes
    - Fonctions et propriétés des composantes

C’est Quoi Architecture Logicielle? (4/4)

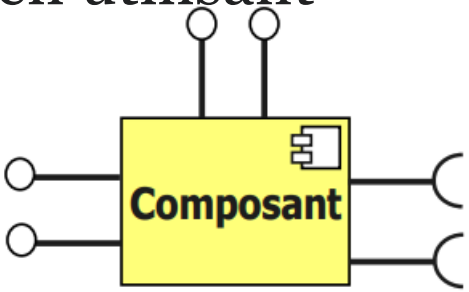
L’**architecture logicielle = Abstraction logiciel**

- Une architecture logicielle est une **représentation abstraite** d’un système.
- Elle peut être exprimée essentiellement à l’aide de **composants logiciels** en interaction via des **connecteurs**



Composants Logiciels

- Un composant est un **module logiciel** répondant à un **besoin fonctionnel**/**applicatif** (application, bibliothèque, module, etc.) ou un **entrepôt de données**
- Un composant est identifié par son **nom q**ui indique son **rôle** dans le système
- Les composants communiquent entre eux en utilisant des **ports/ou interfaces**

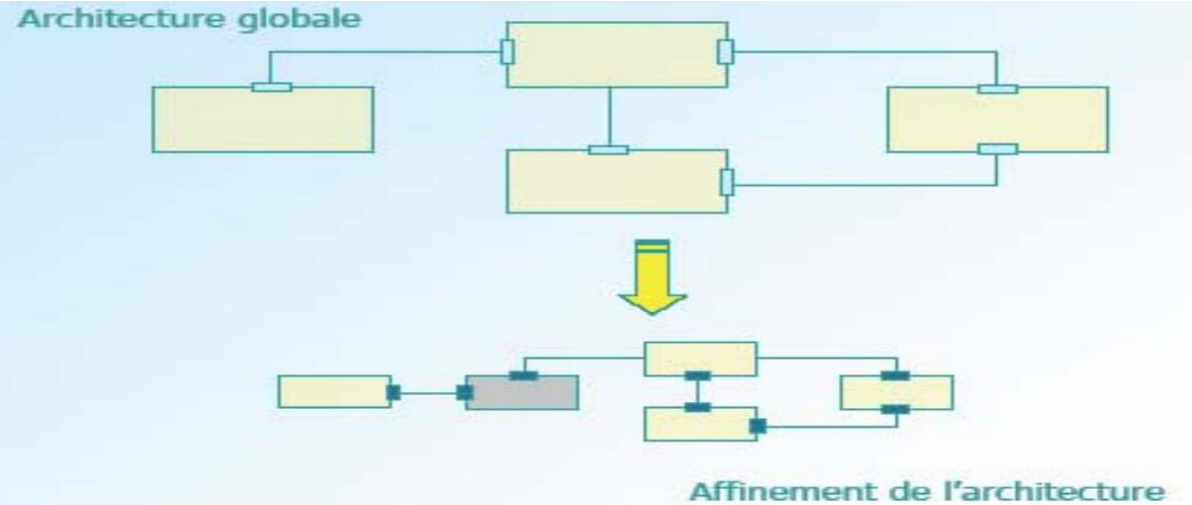


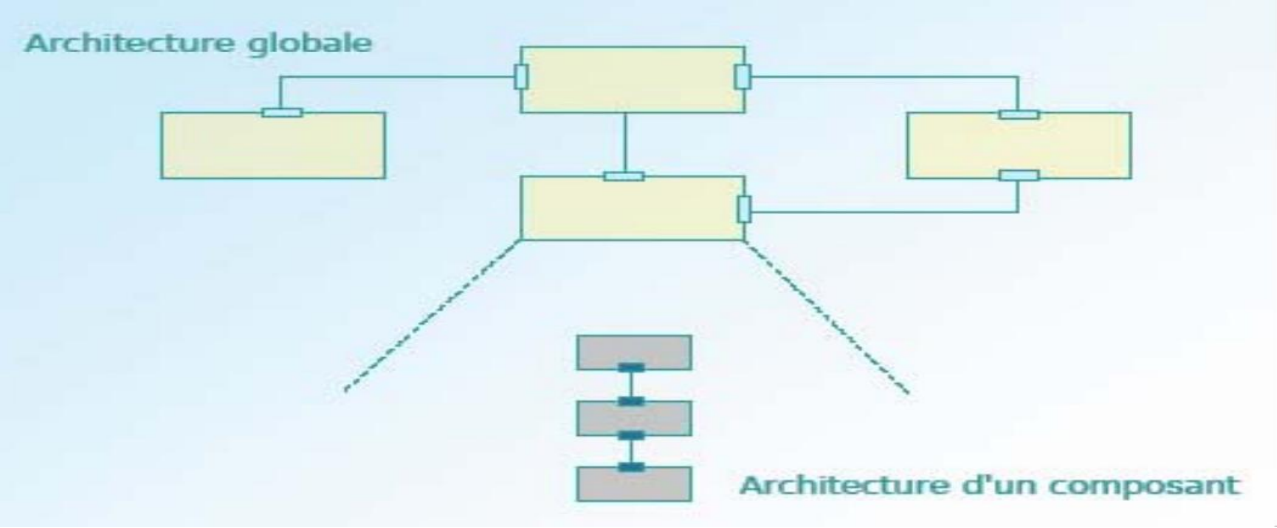
- **Ne pas confondre**
  - Spécification
  - Réalisation

Connecteurs

- Le **connecteur** modélise une **interaction** entre deux **composants**
- C'est donc un **composant spécialisé** dans **l'interaction**, ne répondant pas directement à un besoin fonctionnel/ applicatif
- Il assure l'i**nteractions** entre les **composants**
- Peuvent être de **complexité variable:**
  - **Interaction simple** telle que *appel de méthode*
  - Ou **Interaction complexe** (exemple le *protocole http*)
- Un connecteur propose des rôles aux composants (exemple producteur, consommateur, etc.)
- 

Une succession d'Abstractions

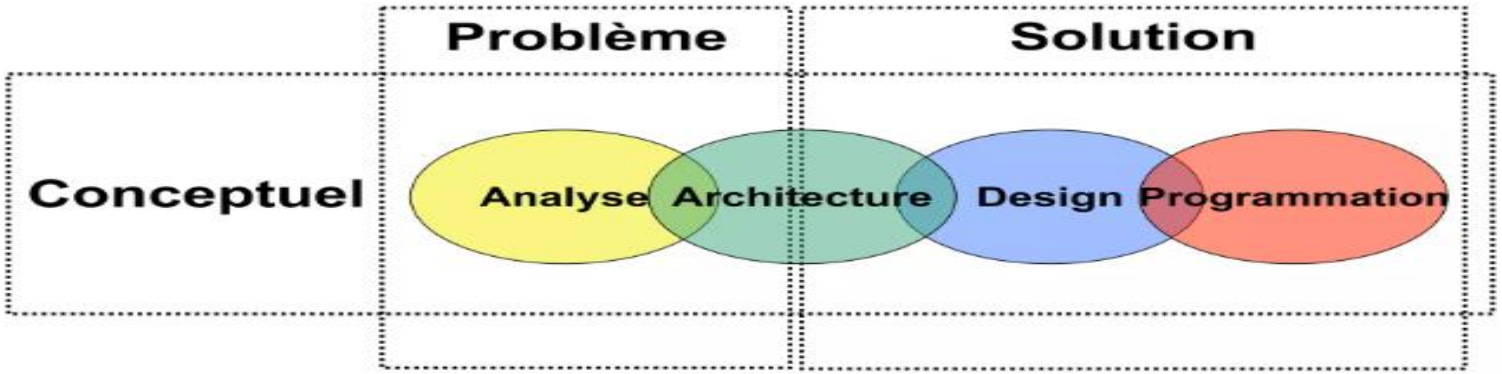


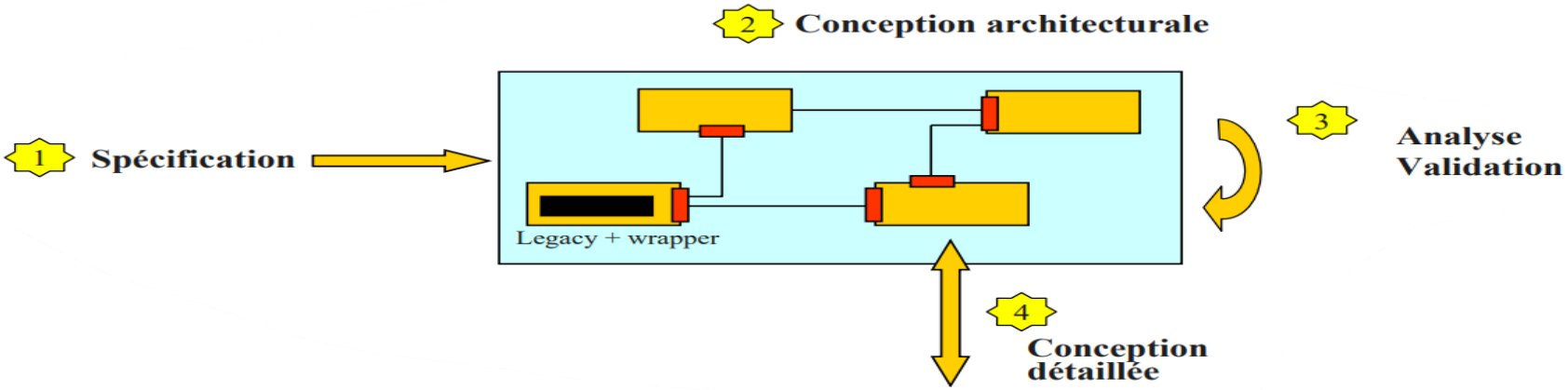


Positionnement

L'architecture = première étape de conception

- **réduire** la complexité du système abordé en le **structurant** en **composants logiciels**





Intérêt (1/4):
Enfin des Réponses

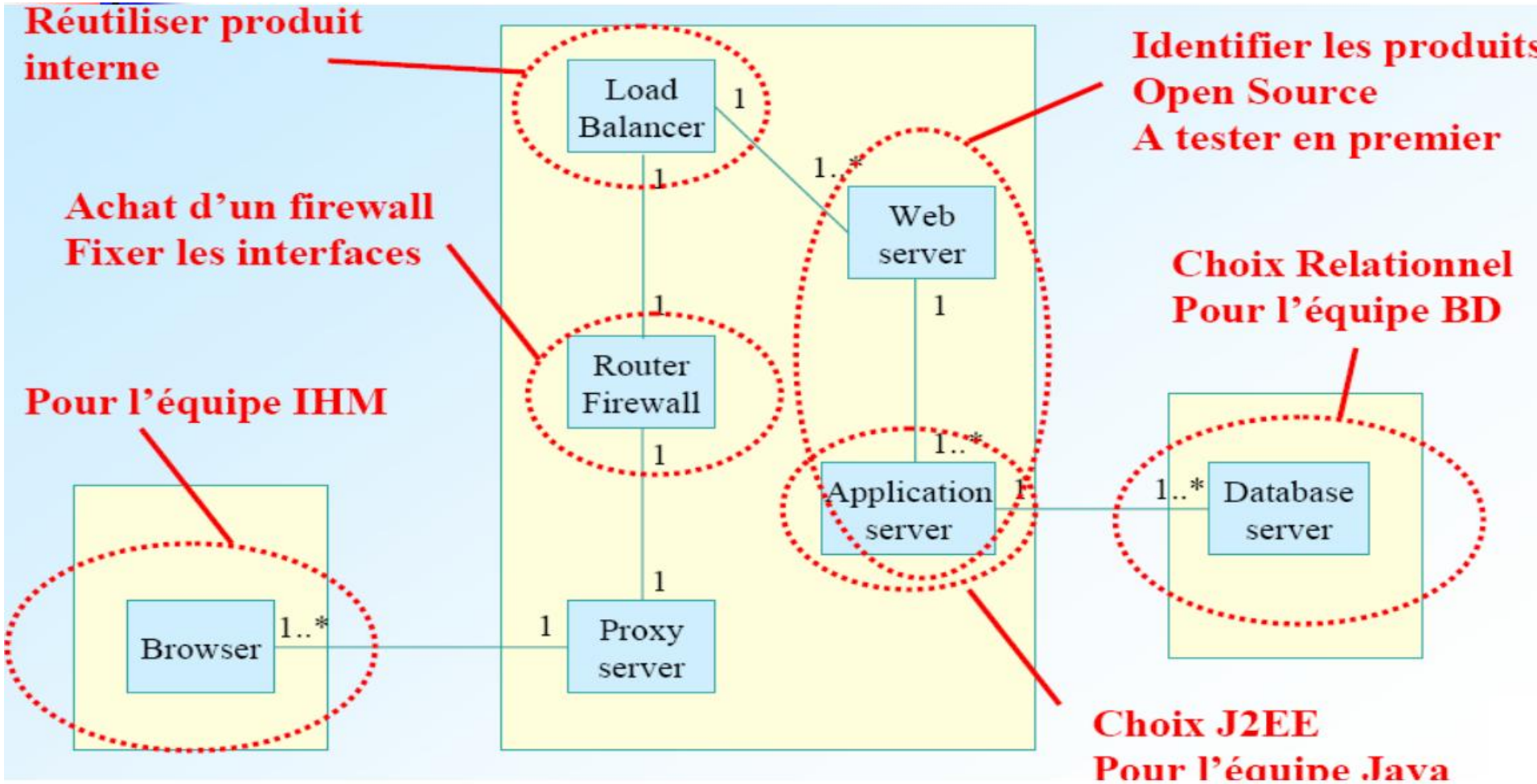


- Première étape de conception
- Permet de réfléchir et répondre aux questions :

- **Où développer ?**
- **Comment développer ?**
- **Quelles équipes, quelles techno ?**
- **Quel coût ?**

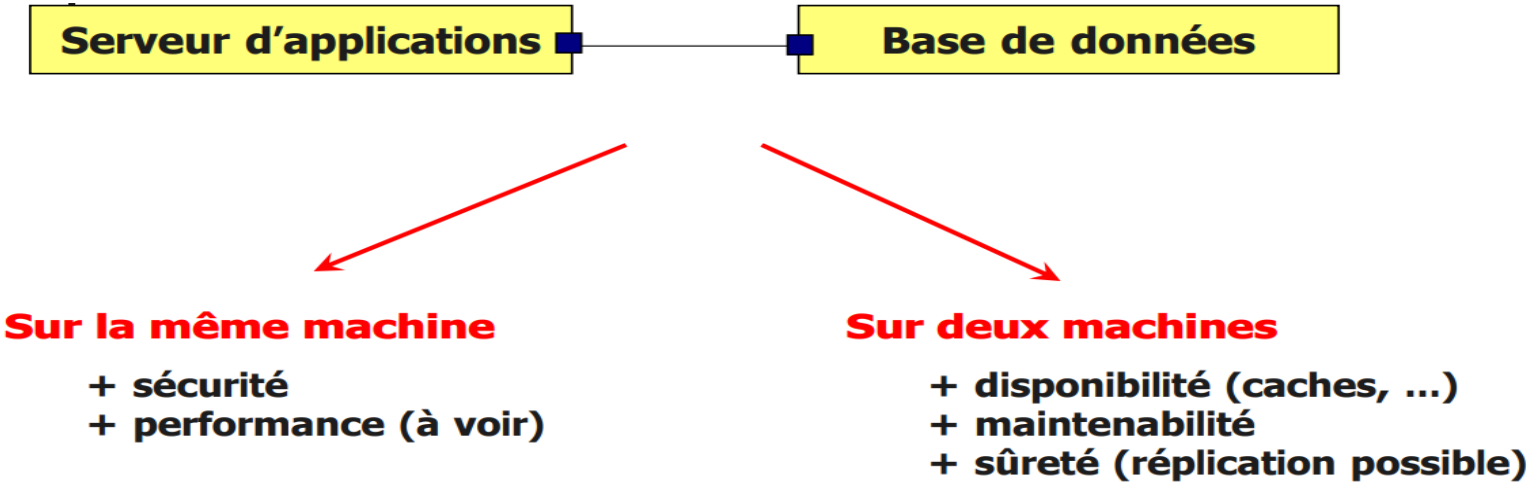
Intérêt (2/4):

Exemple d'Architecture



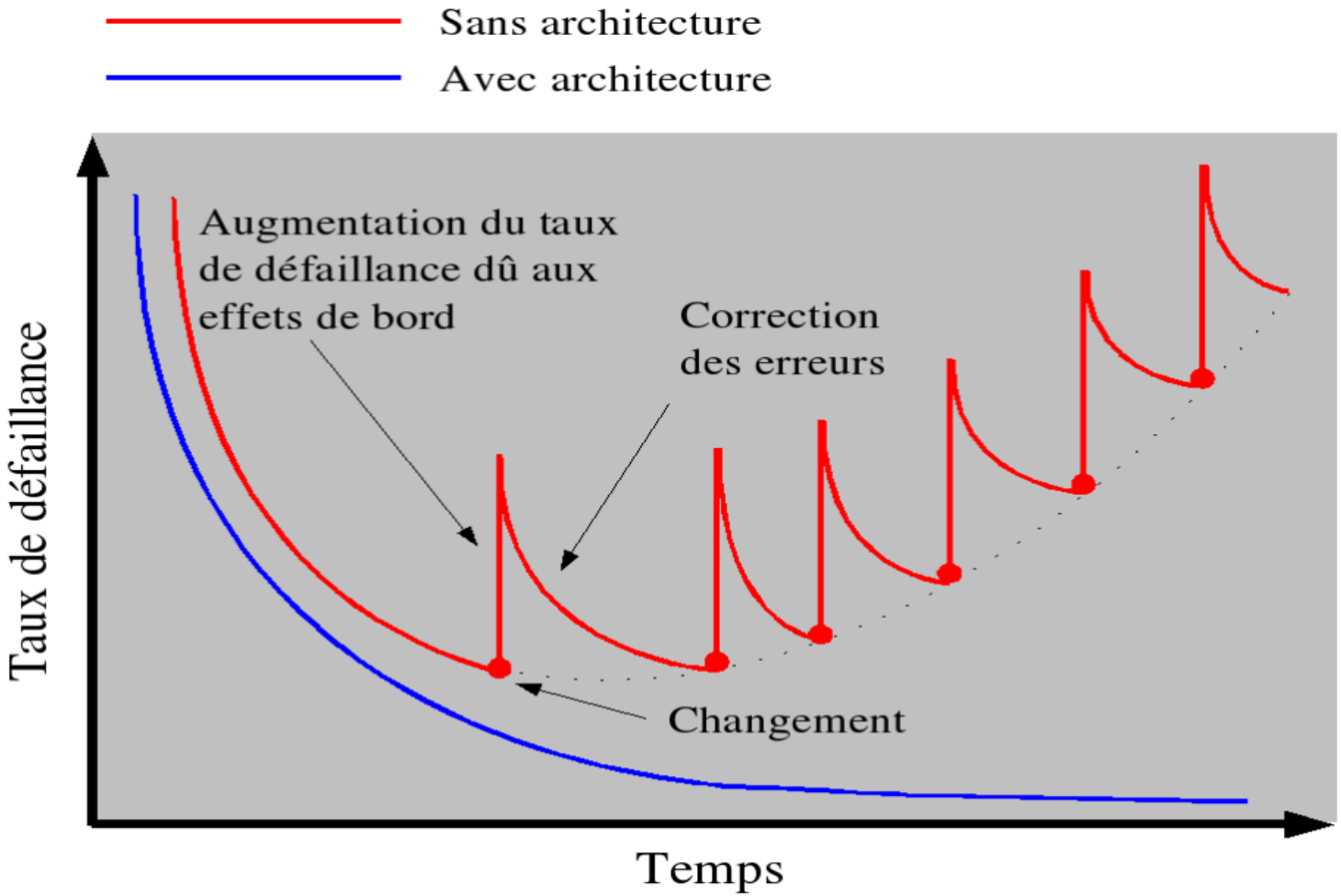
Intérêt (3/4):
Impact sur la Qualité Logicielle

- L'architecture a une forte **influence** sur les propriétés finales d'un système
- La structuration architecturale **favorise** ou **pénalise** les propriétés non fonctionnelles telles que
  - Performance, Sûreté, Sécurité, Disponibilité, Maintenabilité,...



Un premier niveau de **compromis** se fait au niveau architectural.

Intérêt (4/4):
Impact sur la Qualité Logicielle (suite)



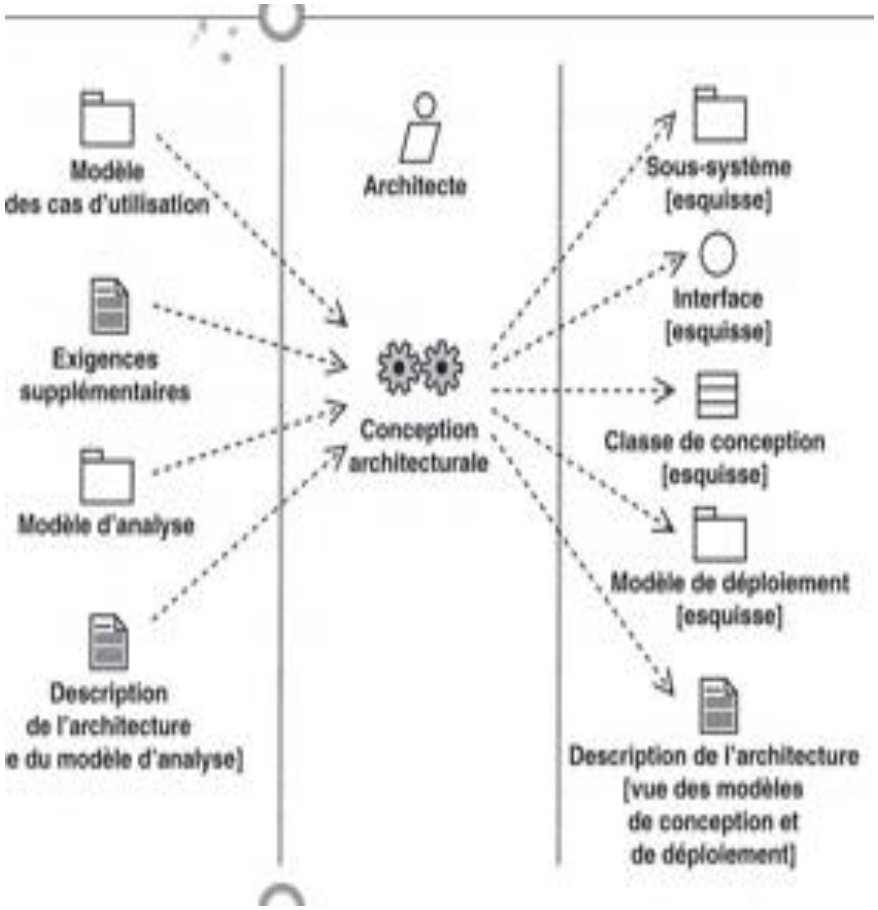
Source *:commons.wikimedia.org [consulté le 29/08/2023]*

Architecture Vs Conception

- L'**architecture** est **une forme de conception**
- Les **conceptions ne sont pas toutes architecturales**
  - Plusieurs détails de bas niveau ne sont pas spécifiés dans une architecture
    - Ex: Le choix d'une structure de données
    - Ex: Le choix d'un algorithme
- L'architecture **ne définit pas** une **implémentation** du logiciel
- Mise **plutôt** sur la **structure** et **moins** sur **le domaine d'application**

Concevoir l'Architecture

- Concevoir l'architecture signifie:
  - Estimer les besoins (physiques) :
    - Performance, concurrence, gestion des données, IHM, robustesse
  - Organiser le système en sous systèmes
  - Choisir un style architectural
  - Intégrer l'existant
  - Choisir un modèle de déploiement



Présentation d'un Architecte Logiciel (1/6)

Pourquoi un architecte? (maison)

- Satisfaire les besoins du propriétaire
- Assurer la longévité de la maison
- Planifier pour permettre les extensions possibles
- Faciliter l'entretien et le confort
- Contenir les coûts
- Guider les travailleurs qui vont la construire



Présentation d'un Architecte Logiciel (2/6)

Pourquoi un architecte? (maison)

- Besoins… Description du mandat
- Longévité de la maison… Robustesse
- Extensions possibles… Modificabilité
- Entretien… Maintenabilité
- Confort… Utilisabilité
- Contenir les coûts … Gestion du projet
- Guider les travailleurs…. Devis/plan du projet



Présentation d'un Architecte Logiciel (3/6)

Pourquoi un architecte logiciel?

- Assurer la stabilité et la robustesse du système
- Permettre une maintenance relativement facile
- Permettre les évolutions futurs
- Contenir les coûts
- Minimiser les compilations
- Guider les programmeurs dans leurs effo de développement



Présentation d'un Architecte Logiciel (4/6) : Définition

- **L'architecte logiciel**
  - c'est la personne qui **propose des solutions** qui sont mises en œuvre par l'équipe de conception et de développement.
  - est un **développeur logiciel expérimenté**, **responsable** du **design** de la **planification** et de la **méthodol** utilisées **pour le développement d'u produit**.



Présentation d'un Architecte Logiciel (5/6): Responsabilité

- En tant qu'un architecte logiciel, vous serez en charge de :
  - Communiquer avec des clients et des parties prenantes
  - Coordonner une équipe de développement
  - Evaluer la portée d'un projet
  - Analyser une architecture logicielle existante
  - Définir la structure et le comportement d'application et leurs interactions
- En tant qu'un architecte logiciel, vous pouvez prétendre au énumérations suivantes annuelles brutes (ces salaires peuvent varier selon les régions):
  - Moyenne pour un architecte logiciel : 82.500 € an(6875 € brut/mois). *source **glassdoor.fr**[consulter le 29/08/2023]*



Présentation d'un Architecte Logiciel (6/6):

Compétences requises

Les compétences requises pour être un bon architecte logiciel sont variées et couvrent un large éventail de domaines. Elles couvrent:

- Compétences conceptuelles
  - Maitrise plusieurs approches et méthodes de conceptions telles que orienté objet, (UML), Merise, Formelle (B, Z), etc.
  - Capable de créer et interpréter des diagrammes d'architecture, tels que des diagrammes de composants, de déploiement, de séquence, etc.
- Compétences liées au domaine métier
  - Assiste à la phase de collecte des besoins
  - Discute avec l'utilisateur final à propos des règles de gestion de l'entreprise



- Compétences technologiques:
  - Maîtrise des langages de programmation, des Framework, des bibliothèques et des technologies liées au développement logiciel.
- Etc.



Institut Supérieur d'Informatique et de Mathématiques de Monastir

Partie 2: Vues Architecturales

Contenu de la Partie 2

1. Modélisation d'une Architecture Logicielle
2. L'Architecture Logicielle selon le Modèle Conventionnel
3. L'Architecture Logicielle selon le Modèle de Cruchten
4. Les Cinq Vue du Modèle de Cruchten

Modélisation d'une Architecture Logicielle(1/3)

- Description d'un système complexe selon la formule de Perry et Wolf :
  architecture = éléments + formes + motivations

- Les **éléments** peuvent **varier** selon :
  - **Niveau de granularité**
    - **tailles** (lignes de code, procédures ou fonctions, modules ou classes, paquetages ou couches, applications ou systèmes informatiques)
  - **Raffinement**
    - ébauche, solution à améliorer ou solution finale
  - **Niveau d'abstraction**
    - idées ou concepts, classes ou objets, composants logiciels
  - **Temporalité**
    - une existence limitée dans le temps
  - **localisation**
    - une existence limitée dans l'espace

Modélisation d'une Architecture Logicielle(2/3)

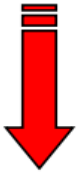
Description d'un système complexe selon la formule de Perry et Wolf :
architecture = éléments + formes + motivations

2. **Formes :**
   - Les formes se réfèrent à la manière dont les éléments sont organisés et interagissent les uns avec les autres. Cela concerne l'architecture physique et logique du système.
     - Par exemple, une architecture en couches, en microservices, ou en monolithique représente différentes formes.
   - La manière dont les éléments sont structurés et connectés détermine la forme architecturale globale.
3. **Motivations :**
   - Les motivations représentent les raisons sous-jacentes qui guident les décisions architecturales.
   - Ces motivations peuvent inclure des objectifs spécifiques tels que la performance, la scalabilité, la sécurité, la facilité de maintenance, etc.
   - Comprendre les motivations permet de justifier les choix architecturaux et de garantir qu'ils répondent aux besoins du système et de ses utilisateurs.

Modélisation d'une Architecture Logicielle (3/3)

Points de vue variés pour le même système d'entreprise

Architecture = composants + connecteurs



Représentation = ensemble de vues

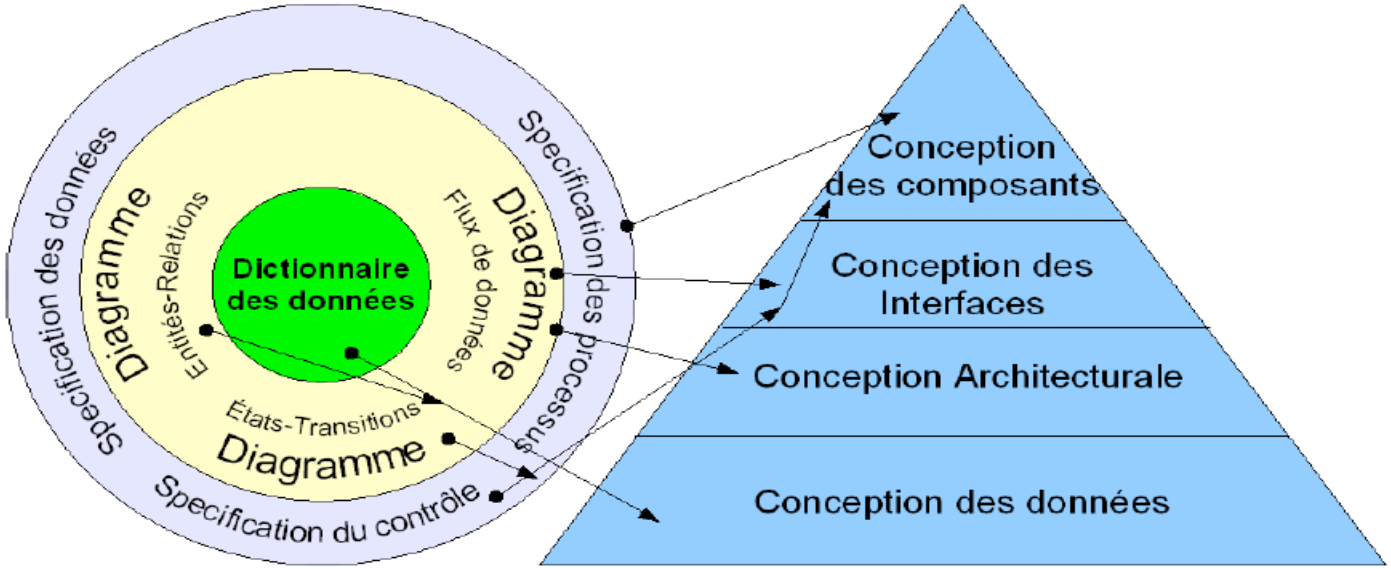






L'Architecture Logicielle selon le Modèle Conventionnel (1/2)

- Le **modèle conventionnel** donne des **directives** pour la **conception de systèmes logiciels** selon deux parties:
  - les spécifications systèmes à gauche
  - et les activités de conception à droite

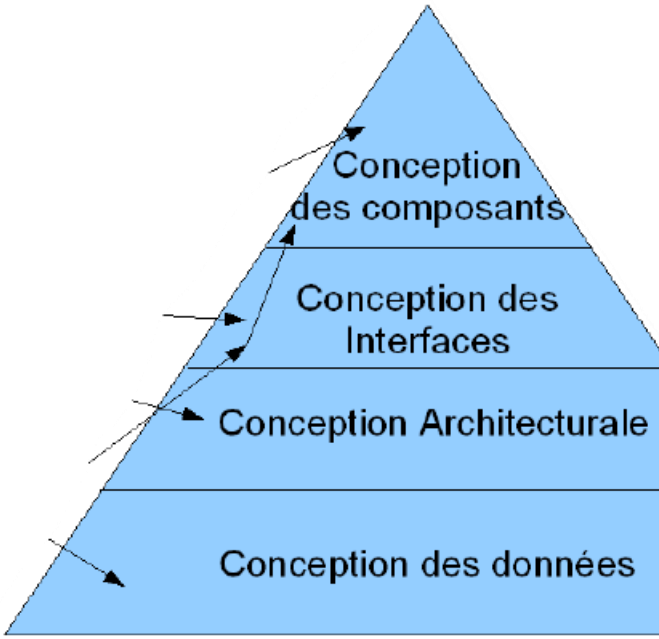


L'Architecture Logicielle selon le Modèle Conventionnel (2/2)

- **L'architecture logicielle traditionnelle** (selon le modèle conventionnel) **implique au moins quatre vues distinctes** :
  - architecture des données,
  - architecture fonctionnelle/modulaire,
  - architecture des interfaces utilisateurs
  - et architecture détaillée des modules.

Problèmes:

- **Pyramide symbolisant** la **construction progressive** de **chaque couche** basée sur la précédente.
- Le **modèle Conventionnel impose la séparation entre** données, traitements et présentation
- **Difficulté de distinction** entre l'analyse et la conception



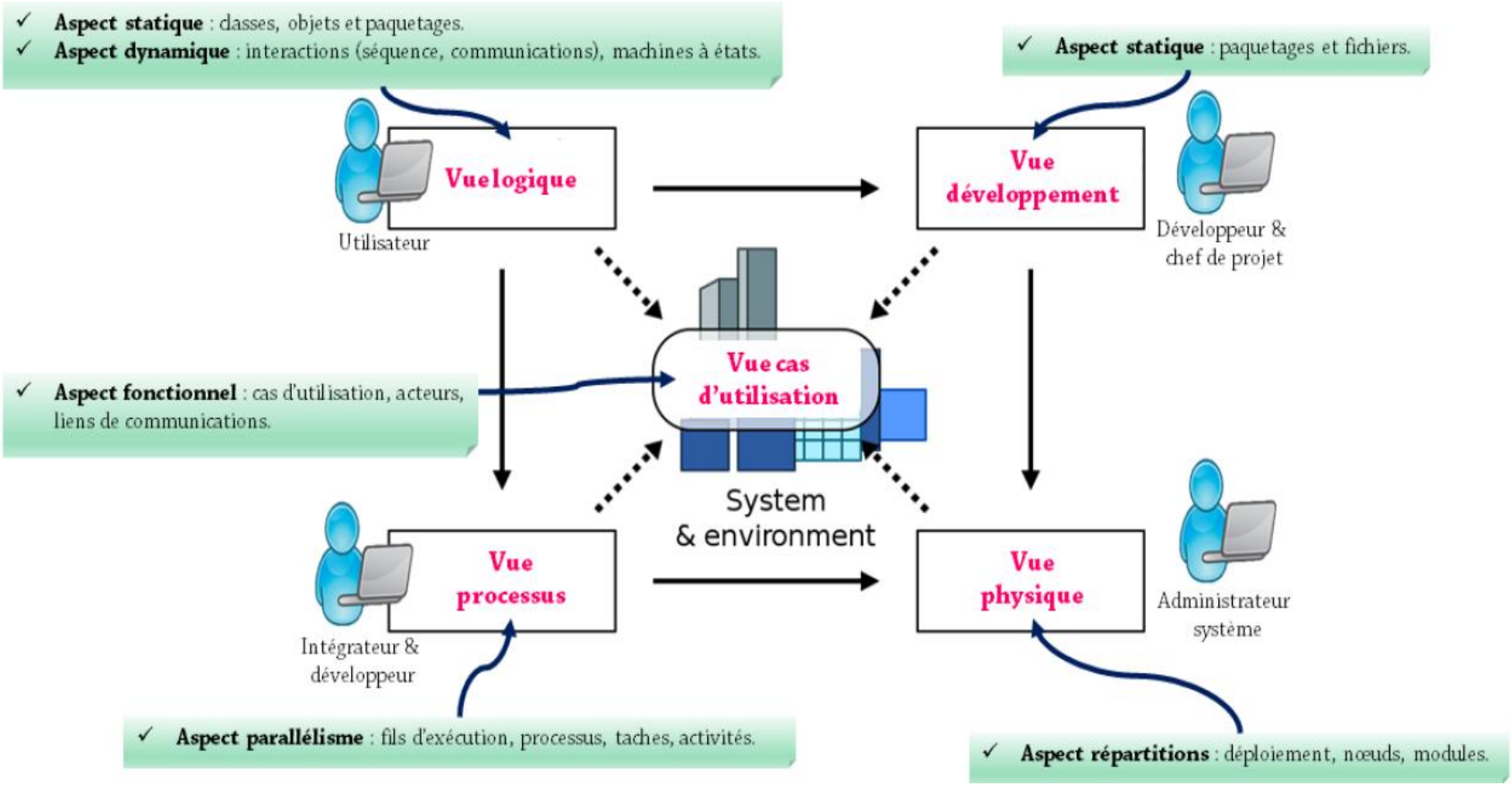
L'Architecture Logicielle selon le Modèle de Cruchten :

Définition

- Le **modèle de Cruchten** est appelé aussi **modèle des 4+1 Vues:**
  - adopté dans **l'Unified Process**,
  - Le **modèle d'analyse** devient une vue dite **vue de cas d'utilisation:**
    - **Constitut** le **lien** des **autres vues**,
    - et **motive** la **création** de tous **les diagrammes architecturaux**

  - permet d'organiser la description du système en **plusieurs vues complémentaires**, chacune **présentant** le **système selon un point de vue différent.**
  - **L'utilisation de vues** permet de:
    - **traiter séparément les intérêts** des divers groupes d'intervenants (architectes, utilisateurs, développeurs, chefs de projets, etc.)
    - **Mieux séparer** les **préoccupations fonctionnelles** (le domaine d'application ou métier ciblé par le système et les **préoccupations non fonctionnelles** (les propriétés techniques telles que la sûreté de fonctionnement).

Les Cinq Vue du Modèle de Cruchten (1/14)



Les Cinq Vue du Modèle de Cruchten (2/14):

Vue de Cas d'utilisation

Définition

- La **vue des cas d'utilisation** est **un modèle d'analyse**.
- Un cas d'utilisation est défini comme un **ensemble de scénarios d'utilisation**,
  - chaque **scénario** représentant une **séquence d'interaction** des utilisateurs (acteurs) **avec le système**.
- **L'intérêt des cas d'utilisation** est **de piloter l'analyse** par les **exigences des utilisateurs**.
  - Ceux-ci se sentent concernés car ils peuvent facilement comprendre les cas d'utilisation qui les concernent.

Les Cinq Vue du Modèle de Cruchten (3/14):

Vue de Cas d'utilisation (suite)

Définition (suite)

- La vue de cas d'utilisation se concentre sur la **cohérence** en présentant des **scénarios d'utilisation** qui mettent en œuvre les éléments des quatre autres vues.

- Elle **valide** en quelque sorte les **autres vues** et **assure** la **cohérence globale**.

- Elle est construite en premier, juste après l'établissement du cahier des charges, pour fixer les contours du système à réaliser avec ses fonctionnalités appelées, dans la terminologie UML, des cas d'utilisation.

Les Cinq Vue du Modèle de Cruchten (4/14):

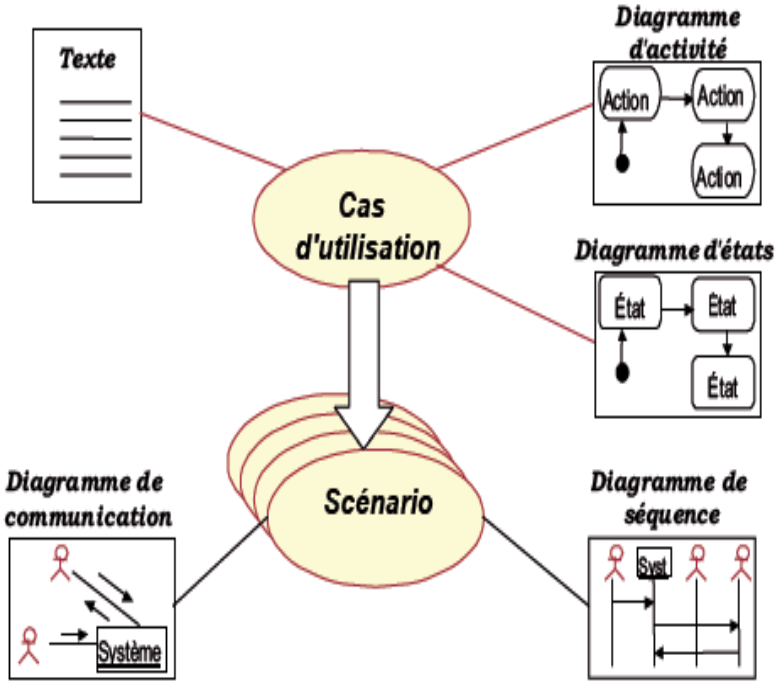
Vue de Cas d'utilisation (suite)

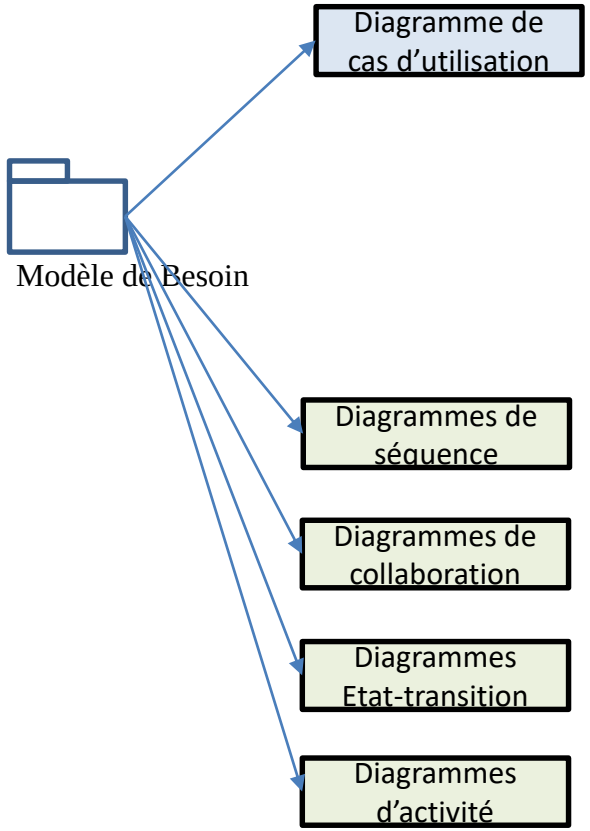
Diagrammes utilisés

- Cette méthode permet donc d'aider à formaliser les véritables besoins et attentes des utilisateurs ; leurs critiques et commentaires étant les briques de la spécification du système.
- L'ensemble **des cas d'utilisation** du logiciel en cours de spécification est **représenté par un diagramme de cas d'utilisation**,
  - chacun des scénarios de celui-ci étant décrit par un ou **plusieurs diagrammes dynamiques** :
    - diagrammes d'activités,
    - diagramme de séquence,
    - diagrammes de communication
    - Ou diagramme d'états-transitions.

Les Cinq Vue du Modèle de Cruchten (5/14):
Vue de Cas d'utilisation (suite)

Diagrammes utilisés (suite)





Les Cinq Vues du Modèle de Cruchten (6/14): Vue logique

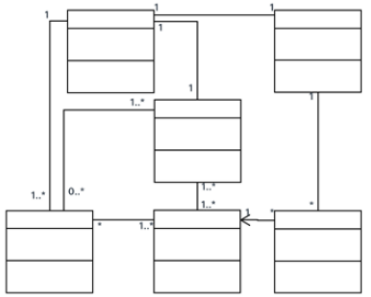
Définition

- La **vue logique constitue** la **principale description architecturale** d'un système informatique et beaucoup de petits projets se contentent de cette seule vue.
- Elle décrit les **aspects statiques** et **dynamiques** d'un système en termes de classes, d'objets, de connexions et de communications.
- Elle permet de décomposer le système en abstractions et **constitue le coeur de la réutilisation**.
- En effet, l'architecte informatique récupérera un maximum de composants des différentes bibliothèques et cadriciels (framework) à sa disposition.
- Une **recherche active** de composants libres et/ou commerciaux pourra également être envisagée.

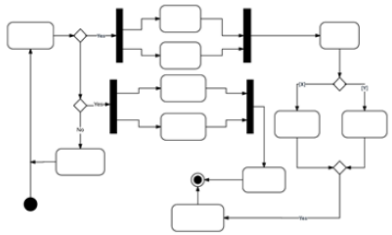
Les Cinq Vues du Modèle de Cruchten (7/14):
Vue logique (suite)

Diagrammes utilisés

- La **vue logique** est représentée, principalement, par:
- **des diagrammes statiques**
  - de classes
  - et d'objets



- **enrichis de descriptions dynamiques :**
  - diagrammes d'activités, de séquence,
  - diagrammes de communication
  - ou d'états-transitions.



Les Cinq Vues du Modèle de Cruchten (8/14):

Vue de processus

Définition

- La **vue logique de processus** capte **les aspects de concurrence** et de **synchronisation,** et les **décompose** en **flux d'exécution** (processus, fil d'exécution, etc.).

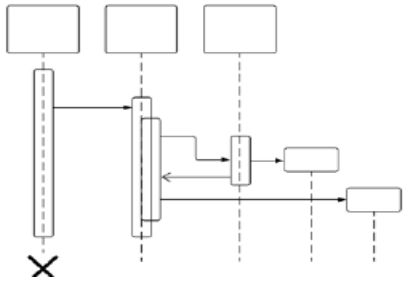
- Elle se rapporte aux **objets actifs** et **aux interactions.**

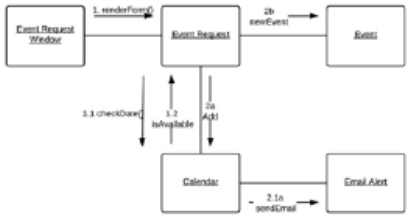
- Cette vue est **très importante** dans l'environnement **multitâches;** montre:
  - La **décomposition** du système **en** terme de **processus (tâches)**
  - Les **interconnexions** entre les **processus** (leur communication)
  - La **synchronisation** et la communication des activités parallèles (threads)

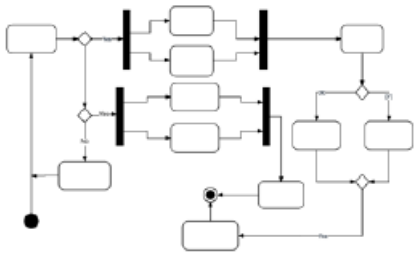
Les Cinq Vues du Modèle de Cruchten (9/14): Vue de processus (suite)

Diagrammes utilisés

- La **vue de processus se concentre sur le comportement de l'execution du système**
- Les diagrammes UML qui peuvent utilisés incluent:
  - diagrammes de séquence,
  - diagrammes de communication
  - Diagramme d'activité.







Les Cinq Vues du Modèle de Cruchten (10/14): Vue de développement

Définition

- La **vue de développement** dite aussi **vue des composants, vue de réalisation, vue d'implémentation illustre** le **système de point de vue d'un programmeur** et s'intéresse à la gestion des logiciels
- Elle représente l'organisation **statique** des modules (exécutable, code source, paquetage, etc.) dans l'environnement

Les Cinq Vues du Modèle de Cruchten (11/14): Vue de développement (suite)

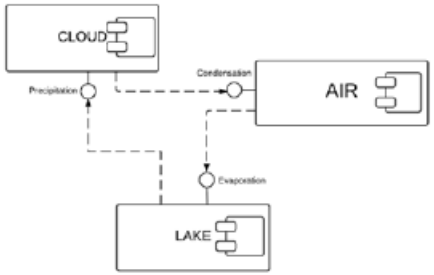
Définition (suite)

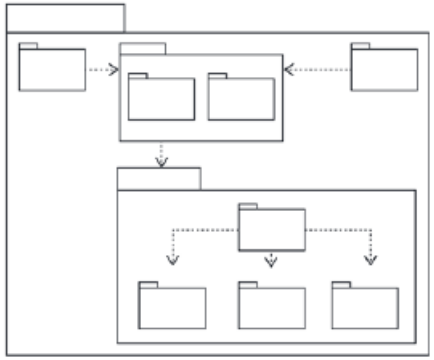
- Cette vue de bas niveau montre:
  - L'allocation des éléments de modélisation dans les modules (fichiers sources, bibliothèques dynamiques, bases de données, exécutables, etc.)
  - En d'autres termes, cette vue **identifie** les **modules** qui réalisent (physiquement) les **classes de la vue logique**
  - L'organisation des composants i.e la **distribution du code** en gestion de la **configuration**, les **dépendances entre les composants**, etc.
  - Les contraintes de développement (bibliothèques externes, …)
  - La vue des composants **montre** aussi **l'organisation des modules en « sous-systèmes»**, les **interfaces** des sous-systèmes et **leurs dépendances** (avec d'autres sous-systèmes ou modules)

Les Cinq Vues du Modèle de Cruchten (12/14):
Vue de développement (suite)

Diagrammes utilisés

- Les diagrammes UML utilisés pour représenter la vue de développement incluent :
  - Le diagramme de composants
  - Le diagrammes de package





Les Cinq Vues du Modèle de Cruchten (13/14):

Vue physique

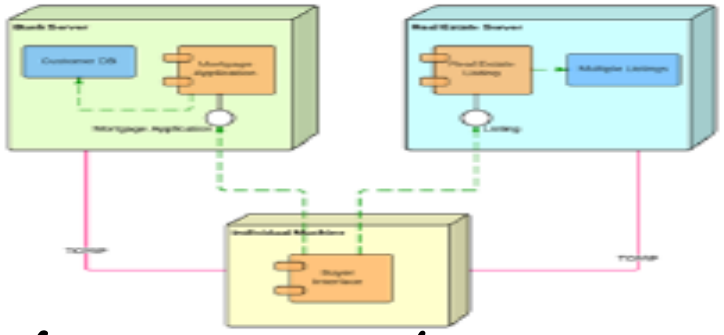
Définition

- La **vue physique** dite aussi vue de **déploiement** représente le **système dans son environnement** d'exécution.
- Elle décrit les différents ressources matérielles et l'implantation logicielle tenant compte de ces ressources.
- Cette nœud est **très utile** dans les **environnements distribués**, décrit les ressources **matérielles et la répartition** du système dans ces ressources:
  - La **disposition** et **nature physique** des **matériels**, ainsi que leurs **performances**
  - L'**implantation** des **modules principaux** sur les nœuds du réseau
  - Les **exigences en terme de performances** (temps de réponse, tolérance au fautes et pannes, etc.)

Les Cinq Vues du Modèle de Cruchten (14/14):
Vue physique(suite)

Définition (suite)

- Cette vue représente le système de point de vue ingénieur système.
- Elle s'intéresse au **topologie des composants logiciels** sur la **couche physique** ainsi qu'aux connexions physiques entre ces composants.



Diagrammes utilisés

- Les **diagrammes UML utilisés** pour représenter cette vue incluent le **diagramme de déploiement**

Fin

aljia.bouzidi95@gmail.com